

2303a52416

batch=35

Assignment =13.1

Task 1: Removing Code Duplication

Code

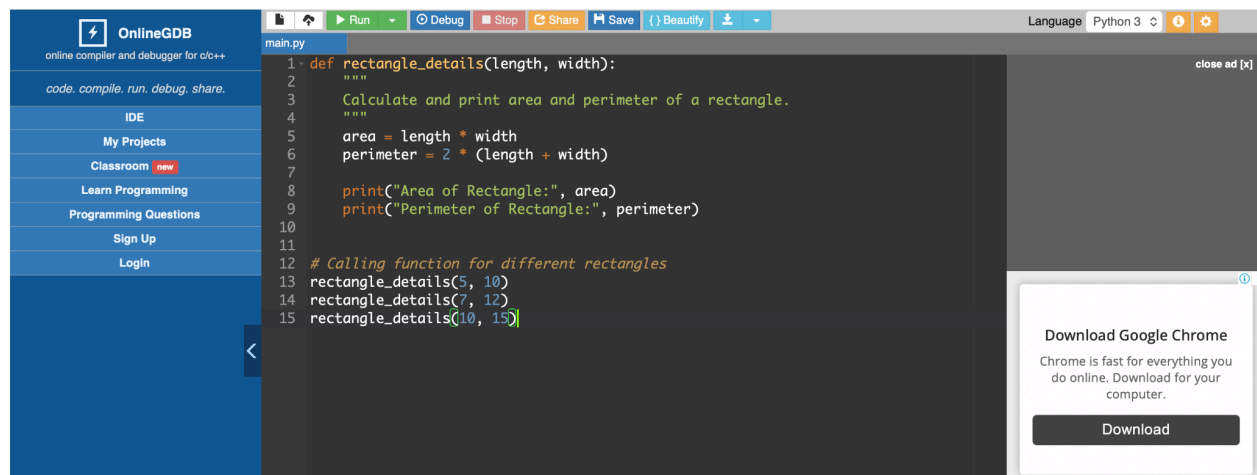
```
def rectangle_details(length, width):  
    """  
    Calculate and print area and perimeter of a rectangle.  
    """  
    area = length * width  
    perimeter = 2 * (length + width)  
  
    print("Area of Rectangle:", area)  
    print("Perimeter of Rectangle:", perimeter)
```

# Calling function for different rectangles

rectangle\_details(5, 10)

rectangle\_details(7, 12)

rectangle\_details(10, 15)



## Explanation

- The legacy code repeated the same calculations multiple times.
- A function **rectangle\_details()** is created.
- The function calculates **area and perimeter**.
- We call the function with different values.
- This improves **code reuse and readability**.

# Task 2: Optimizing Loops and Conditionals

Code

```
import time
```

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
```

```
# Before optimization
```

```
start = time.time()
```

```
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f'{s} is in the list')
    else:
        print(f'{s} is not in the list')
```

```
end = time.time()
```

```
print("Execution Time (Old Code):", end - start)
```

```
# After optimization using set
```

```
start = time.time()
```

```
name_set = set(names)
```

```
for s in search_names:
    if s in name_set:
        print(f'{s} is in the list')
    else:
        print(f'{s} is not in the list')
```

```
end = time.time()
```

```
print("Execution Time (Optimized Code):", end - start)
```

The top screenshot shows the 'Before optimization' code in the OnlineGDB IDE. It uses nested loops to find common elements between two lists. The code is as follows:

```
1 import time
2
3 names = ["Alice", "Bob", "Charlie", "David"]
4 search_names = ["Charlie", "Eve", "Bob"]
5
6 # Before optimization
7 start = time.time()
8
9 for s in search_names:
10     found = False
11     for n in names:
12         if s == n:
13             found = True
14     if found:
15         print(f"{s} is in the list")
16     else:
17         print(f"{s} is not in the list")
18
19 end = time.time()
20 print("Execution Time (Old Code):", end - start)
21
22
23 # After optimization using set
24 start = time.time()
```

The bottom screenshot shows the 'After optimization using set' code. It converts the list to a set and uses a single loop for lookup. The code is as follows:

```
13     found = True
14     if found:
15         print(f"{s} is in the list")
16     else:
17         print(f"{s} is not in the list")
18
19 end = time.time()
20 print("Execution Time (Old Code):", end - start)
21
22
23 # After optimization using set
24 start = time.time()
25
26 name_set = set(names)
27
28 for s in search_names:
29     if s in name_set:
30         print(f"{s} is in the list")
31     else:
32         print(f"{s} is not in the list")
33
34 end = time.time()
35 print("Execution Time (Optimized Code):", end - start)
```

## Explanation

- Original code used **nested loops (slow)**.
- Optimized code converts list to **set**.
- **Set lookup is  $O(1)$**  which is faster.
- Execution time is compared using **time module**.

## Task 3: Extracting Reusable Functions

Code

```
def calculate_total(price):
```

```
    """
```

```
    Calculate total price including 18% tax.
```

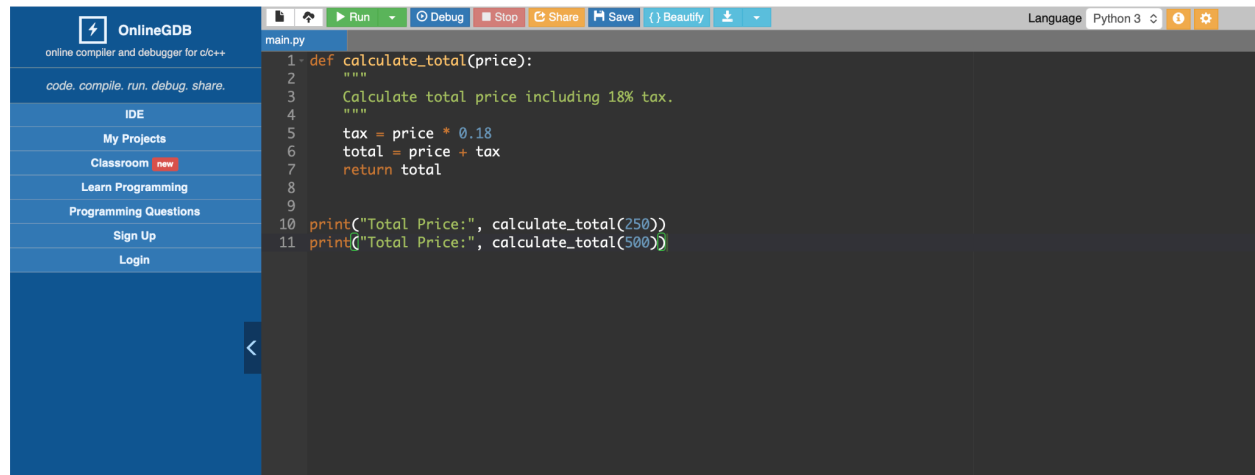
```
    """
```

```
    tax = price * 0.18
```

```
    total = price + tax
```

```
    return total
```

```
print("Total Price:", calculate_total(250))
print("Total Price:", calculate_total(500))
```



The screenshot shows the OnlineGDB web interface. On the left is a sidebar with navigation links: OnlineGDB, code.compile.run.debug.share, IDE, My Projects, Classroom (marked as new), Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a Python file named 'main.py' with the following code:

```
1 def calculate_total(price):
2     """
3     Calculate total price including 18% tax.
4     """
5     tax = price * 0.18
6     total = price + tax
7     return total
8
9
10 print("Total Price:", calculate_total(250))
11 print("Total Price:", calculate_total(500))
```

## Explanation

- Tax calculation was repeated multiple times.
- A function **calculate\_total(price)** is created.
- It returns the total price including tax.
- This makes the code **modular and reusable**.

### Task 4: Replacing Hardcoded Values with Constants

Code

```
# Constants
```

```
PI = 3.14159
```

```
RADIUS = 7
```

```
area = PI * (RADIUS ** 2)
```

```
circumference = 2 * PI * RADIUS
```

```
print("Area of Circle:", area)
```

```
print("Circumference of Circle:", circumference)
```



```
1 # Constants
2 PI = 3.14159
3 RADIUS = 7
4
5 area = PI * (RADIUS ** 2)
6 circumference = 2 * PI * RADIUS
7
8 print("Area of Circle:", area)
9 print("Circumference of Circle:", circumference)
```

## Explanation

- The value **3.14159** was repeated.
- It is replaced with constant **PI**.
- Radius is also defined as constant.
- Constants improve **maintainability**.

## Task 5: Improving Variable Naming

Code

# Base and height of triangle

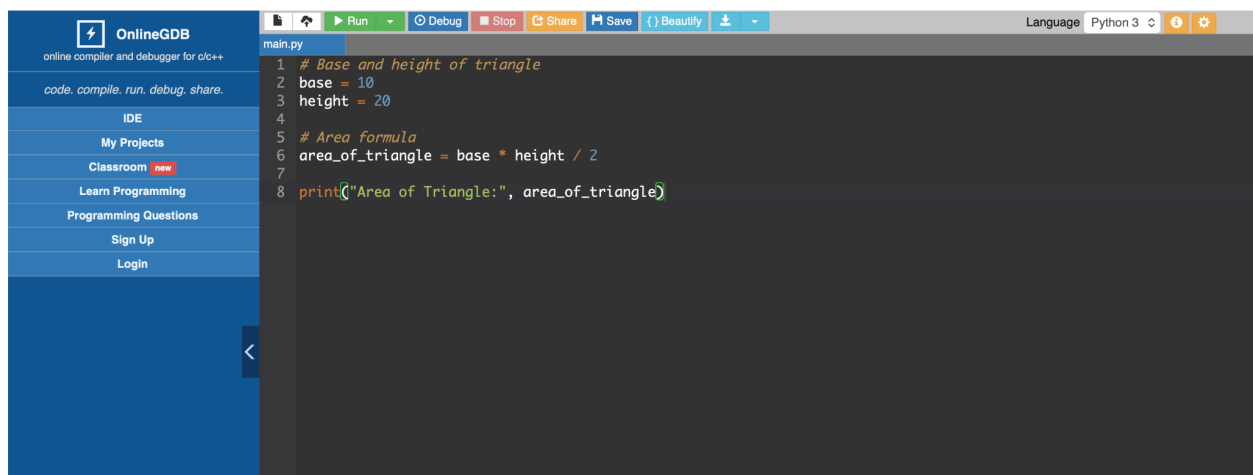
base = 10

height = 20

# Area formula

area\_of\_triangle = base \* height / 2

print("Area of Triangle:", area\_of\_triangle)



```
1 # Base and height of triangle
2 base = 10
3 height = 20
4
5 # Area formula
6 area_of_triangle = base * height / 2
7
8 print("Area of Triangle:", area_of_triangle)
```

## Explanation

- Variables **a**, **b**, **c** were unclear.
- Renamed them to **base**, **height**, **area\_of\_triangle**.
- Added comments for clarity.
- Improves **readability**.

## Task 6: Removing Redundant Conditional Logic

Code

```
def calculate_grade(marks):
```

```
    """
```

```
    Determine grade based on marks.
```

```
    """
```

```
    if marks >= 90:
```

```
        return "Grade A"
```

```
    elif marks >= 75:
```

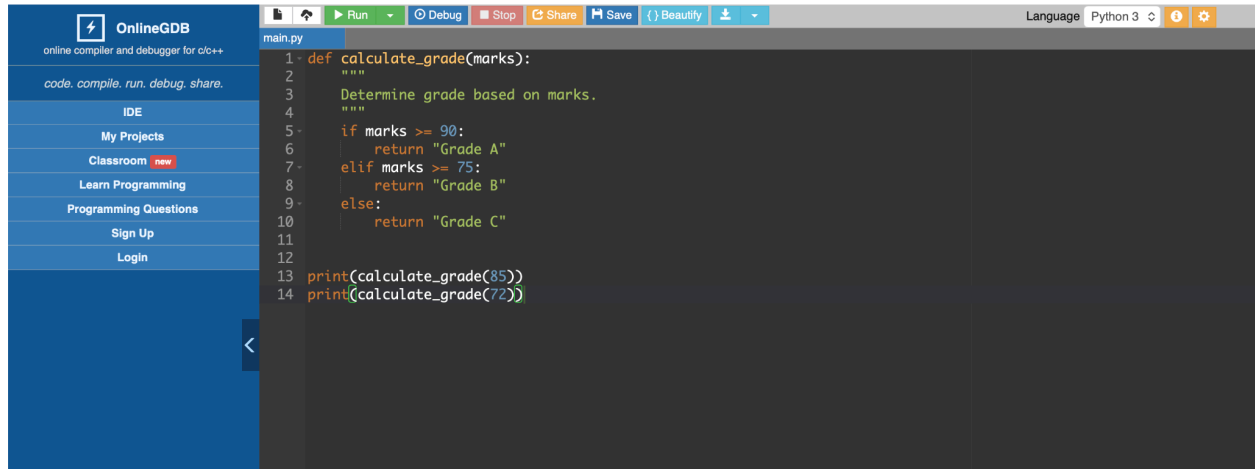
```
        return "Grade B"
```

```
    else:
```

```
        return "Grade C"
```

```
print(calculate_grade(85))
```

```
print(calculate_grade(72))
```



## Explanation

- Same grading logic was repeated.
- Created function **calculate\_grade()**.
- Function returns the grade.
- Avoids repeated code.

### Task 7: Converting Procedural Code to Functions

Code

```
def get_input():
```

```
    return int(input("Enter number: "))
```

```
def calculate_square(num):
```

```
    return num * num
```

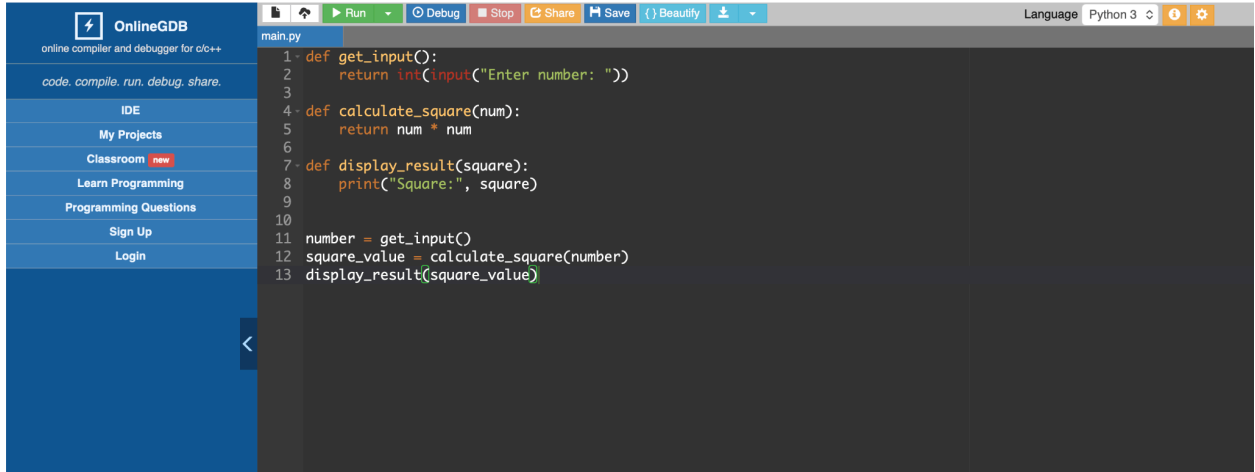
```
def display_result(square):
```

```
    print("Square:", square)
```

```
number = get_input()
```

```
square_value = calculate_square(number)
```

```
display_result(square_value)
```



The screenshot shows the OnlineGDB web interface. On the left is a sidebar with navigation links: OnlineGDB, code.compile.run.debug.share, IDE, My Projects, Classroom (marked as new), Learn Programming, Programming Questions, Sign Up, and Login. The main area displays a Python script in a dark-themed editor. The script defines three functions: get\_input() which takes user input and returns it as an integer; calculate\_square(num) which returns the square of a number; and display\_result(square) which prints the square value. The main execution block calls these functions in sequence: it gets input, calculates the square, and then displays the result. The language is set to Python 3.

```
1- def get_input():
2-     return int(input("Enter number: "))
3-
4- def calculate_square(num):
5-     return num * num
6-
7- def display_result(square):
8-     print("Square:", square)
9-
10-
11- number = get_input()
12- square_value = calculate_square(number)
13- display_result(square_value)
```

## Explanation

Code is divided into **three functions**:

- **get\_input()** → takes input
- **calculate\_square()** → processes data
- **display\_result()** → prints output

This makes the code **modular and structured**.

## Task 8: Optimizing List Processing

Code

```
nums = [1, 2, 3, 4, 5]
```

```
# Using list comprehension
```

```
squares = [n * n for n in nums]
```

```
print(squares)
```



**OnlineGDB**  
online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE

My Projects

Classroom new

Learn Programming

Programming Questions

Sign Up

Login

main.py

1 nums = [1, 2, 3, 4, 5]

2

3 *# Using list comprehension*

4 squares = [n \* n for n in nums]

5

6 print(squares)

Language Python 3